

VSIPL Python Module

Randall Judd

©2026 Randall Judd 2026, all rights reserved.

A non-exclusive, non-royalty bearing license is hereby granted to all persons to copy, modify, distribute and produce derivative works for any purpose, provided that this copyright notice and following disclaimer appear on all copies:

THIS LICENSE INCLUDES NO WARRANTIES, EXPRESSED OR IMPLIED, WHETHER ORAL OR WRITTEN, WITH RESPECT TO THE SOFTWARE OR OTHER MATERIAL INCLUDING, BUT NOT LIMITED TO, ANY IMPLIED WARRANTIES OF MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE, OR ARISING FROM A COURSE OF PERFORMANCE OR DEALING, OR FROM USAGE OR TRADE, OR OF NON-INFRINGEMENT OF ANY PATENTS OF THIRD PARTIES. THE INFORMATION IN THIS DOCUMENT SHOULD NOT BE CONSTRUED AS A COMMITMENT OF DEVELOPMENT BY ANY PARTY.

Introduction

This document describes an encapsulation of a public domain C VSIPL library as a python module. It is intended to be a python starting point for folks who download the distribution. It is not intended as a standalone document. The C VSIPL specification and related documents are needed to understand functionality.

Encapsulating VSIPL library into a python module

I use the publicly available simple wrapper and interface generator tool

SWIG <https://www.swig.org>

to create a C wrapper file **vsip_wrap.c** and a **vsip.py** python module. I include a **vsip.i** SWIG interface file.

There is a **setup.py** file with instructions for installing the **vsip** module. We assume the **libvsip.a** file has been made and is in the standard location in the **jvsip** distribution. The **vsip.h** file also needs to be in the standard location. If you downloaded the distribution from **github** and did a **make** then all those things should be true. Do

python -m pip install .

(don't forget the period at the end) should run SWIG, compile **vsip_wrap.c** and *a few other c files* I have included and link libvsip.a and create and install the **vsip** module.

a few other c files

There is C code in the `vsip\c_src` directory to handle some problems. The **SWIG** encapsulation does not handle macros from the **C VSIP source code** so we fix that in this directory. I also wanted a method with some performance to create python lists from C views and vice versa. That code is contained in this directory and is wrapped and compiled along with the normal VSIP library.

Python VSIP Fundamentals

Using the default build will result in a module called **vsip**. Since C VSIPL function names are not likely to be the same as anything else I generally import them without a name space. A simple example for instance.

```
from vsip import *
def aRamp(start, increment, length):
    vsip_init(None)
    out = []
    a = vsip_vcreate_f(length, VSIP_MEM_NONE)
    vsip_vramp_f(start, increment, a)
    for i in range(length):
        out.append(vsip_vget_f(a, i))
    vsip_valldestroy_f(a)
    vsip_finalize(None)
    return out
aRamp(0, 1, 10)
```

You get [0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0]

I do not know all the differences between C and Python and what work arounds there might be. I am not sure how, or if, you can do the equivalents of `admit`, `release` and `copy from/to user space`; so these may not be useful in the python space.

Notice that the python garbage collection will not work with VSIPL so using out of the box C VSIPL calls require the normal destroy functions be called to avoid memory leaks.

Some functions in VSIPL allow passing a NULL pointer or a pointer to **(void *)0** which is generally considered to be equivalent to passing a NULL pointer. For python this is equivalent to the **None** argument. So for instance to initialize the library in C you do

```
vsip_init((void*)0);
```

in python this is

```
vsip_init(None)
```

VSIPL defines a complex scalar type in the standard way as

```
typedef struct {
    vsip_scalar_f r,i;
} vsip_cscalar_f;
```

To get a complex scalar in C you do

```
vsip_cscalar_f a_cscalar;
```

which would be allocated at compile time or you could do

```
vsip_cscalar_f * a_cscalar_p;
a_cscalar_p = (vsip_cscalar_f*) malloc(sizeof(vsip_cscalar_f));
```

to allocate a complex scalar at run time. You would need to free this to avoid memory leaks. Assuming the former you would use `a_cscalar` like

```
a_cscalar = vsip_cvget_f(a,i);
```

to get a complex scalar from vector `a` at index `i` or

```
vsip_cvput_f(a,i,a_cscalar);
```

to put a complex scalar into position at index `i` for vector `a`.

So this was problematic when I first went to try the VSIP python module. How had swig handled these cases? How do I create a complex scalar?

Turns out that in python you do

```
a_cscalar = vsip_cscalar_f()
```

To get the equivalent to the C complex scalar. The real part is `a_cscalar.r` and the imaginary part is `a_cscalar.i`. This is the same as C.

The python `a_cscalar` also acts like a pointer when passed as a VSIPL pointer. So the put operation above becomes

```
vsip_cvput_f(a,i,a_cscalar)
```

This is not exactly the same as C, but close. All VSIPL typedefs that are based on structures (`vsip_vattr_d`, `vsip_lu_attr_f`, `vsip_scalar_mi`, etc.) operate in the same fashion.

Simple defined data types from the vsip header file which **SWIG** does not handle specially act like regular scalars. For instance `vsip_scalar_f`, `vsip_index`, `vsip_length`, etc. translate into the regular python scalar types of float and int. If you do `a_cscalar = vsip_cscalar_f` Python will complain that `vsip_cscalar_f` is not defined. However `a_scalar=vsip_vget_f(a,i)` or `a_scalar = vsip_vget_d(a,i)` will both just return a python float type.

This is fine as long as the mechanism used in C is a regular variable. But there is no simple python mechanism (AFAIK) to convert these to a pointer to scalar. This causes a problem in some of the selection operations. For instance

```
vsip_index i,
vsip_vview_f *a;
vsip_scalar_f afloat;
/* ...some code ... */
afloat = vsip_vmaxval_f(a,i);
```

This returns the max value in vector a in scalar afloat and the index where the value was found into index i. Although I found a way around this in the swig methodology the result of that was to have these functions return a tuple. I wanted a more VSIPL like solution for this interface. So I created 3 functions in C code (file `indexptr.c`) that define an index pointer, a method to return the index pointer to python, and a method to free the memory associated with the index pointer. So an example in python using this method you do

```
from vsip import *
vsip_vramp_f(0,1,a)
vsip_vput_f(a,4,100)
i=vindexptr()
m=vsip_vmaxval_f(a,i)
indx=vindexptrToInt(i)
vindexfree(i)
```

The three new functions are **vindexptr**, **vindexptrToInt**, and **vindexfree**. Note that `vsip_vmaxval_f` may be called with a null pointer in C if only the max value is required and the index is not needed. In python this would look like

```
m=vsip_vmaxval_f(a,None)
```

Example

In python, using the supplied C VSIPL, it is easy to forget (or ignore) C VSIPL requirements for init, finalize, and destroy. For this example I will try to be conformant to VSIPL requirements. Note that a memory leak in python is still a memory leak. This example is the simple vector add (A Simple First Example) in the TASP VSIPL Core Plus book.

```
from vsip import *
N=8 #length of vector
init = vsip_init(None)
def VU_vprint_f(a):
    print(" ".join( ["%4.0f" % vsip_vget_f(a,i) for i in range(vsip_vgetlength_f(a))]))
A = vsip_vcreate_f(N,0)
B = vsip_vcreate_f(N,0)
C = vsip_vcreate_f(N,0)
vsip_vramp_f(0,1,A)
print("A = \n"); VU_vprint_f(A)
```

```

vsip_vfill_f(5,B)
print("B = \n"); VU_vprint_f(B)
vsip_vadd_f(A,B,C)
print("C = \n"); VU_vprint_f(C)
vsip_valldestroy_f(A)
vsip_valldestroy_f(B)
vsip_valldestroy_f(C)
vsip_finalize(None)

```

To run this example save as example1.py

```
> python example1.py
```

A =

```
0 1 2 3 4 5 6 7
```

B =

```
5 5 5 5 5 5 5 5
```

C =

```
5 6 7 8 9 10 11 12
```

Interacting with python Lists

We define functions in C which are used to return a python list object containing the data from a VSIPL vector or matrix. If you are familiar with VSIPL naming conventions the usage should be obvious.

```

cvcopyToList_f(vsip_cvview_f)
cvcopyToList_d(vsip_cvview_d)
vcopyToList_f(vsip_vview_f)
vcopyToList_d(vsip_vview_d)
vcopyToList_i(vsip_vview_i)
vcopyToList_si(vsip_vview_si)
vcopyToList_uc(vsip_vview_uc)
vcopyToList_vi(vsip_vview_vi)
vcopyToList_mi(vsip_vview_mi)
cmcopyToListByRow_f(vsip_cmview_f)
mcopyToListByRow_f(vsip_mview_f)
cmcopyToListByRow_d(vsip_cmview_d)
mcopyToListByRow_d(vsip_mview_d)
mcopyToListByRow_i(vsip_mview_i)
mcopyToListByRow_si(vsip_mview_si)
mcopyToListByRow_uc(vsip_mview_uc)
cmcopyToListByCol_f(vsip_cmview_f)
mcopyToListByCol_f(vsip_mview_f)
cmcopyToListByCol_d(vsip_cmview_d)
mcopyToListByCol_d(vsip_mview_d)
mcopyToListByCol_i(vsip_mview_i)
mcopyToListByCol_si(vsip_mview_si)
mcopyToListByCol_uc(vsip_mview_uc)

```

Example

Converting a previous example we have

```
from vsip import *
def aRamp(start, increment, length):
    vsip_init(None)
    a = vsip_vcreate_f(length,VSIP_MEM_NONE)
    vsip_vramp_f(start,increment,a)
    out = vcopyToList_f(a)
    vsip_valldestroy_f(a)
    vsip_finalize(None)
    return out
aRamp(0,1,10)

returns

[0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0]
```